# Pencils to Pixels: Reimplementing SDEdit

by: Rio Voss-Kernan (rvossker), William Jarvis (wjarvis10), and John Perdue (jperdue21)
CSCI 1470 (Deep Learning), Department of Computer Science, Brown University

Colab Notebook:
https://colab.research.google.com/drive/10538l0ElBKQ7i_InUGN1FpGP0V2WCD50?usp=sharing

## Introduction:

The ability to manipulate digital images with precision and realism has become increasingly important in fields ranging from design to deep learning. One of the key challenges is finding techniques that can modify an image while preserving its underlying structure and coherence. Traditional generative models, including GANs and unconditional diffusion models, struggle to balance faithfulness to the input and realism. To address this, we explore a method called SDEdit, which enables controlled image editing by hijacking the denoising property of diffusion models with an artificially noised input. Our implementation is based on the paper Image Editing with SDEdit: Guided Image Synthesis via Stochastic Differential Equations, and leverages a pretrained diffusion model for image generation of flowers to perform meaningful and accurate edits.

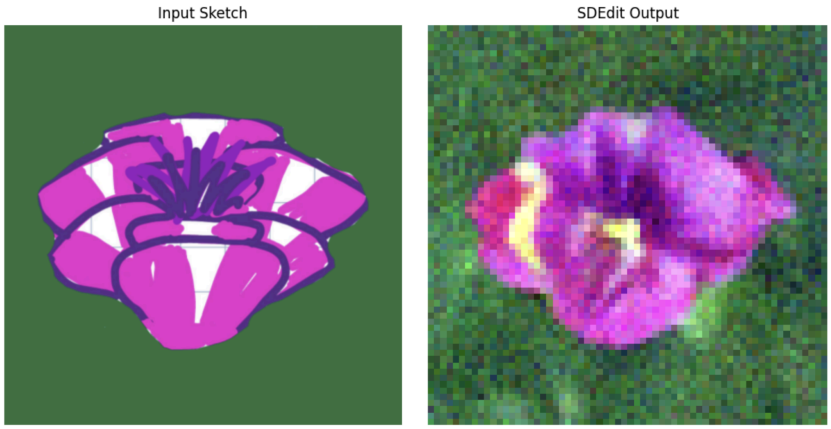



*Figure 1. Full SDEdit generation from sketch noised to  t=0.35*

## Methodology:

We implemented SDEdit by leveraging a pretrained diffusion model trained on the Oxford Flowers 102 dataset, a dataset of 8189 images of various flower types. We added Gaussian noise corresponding to a selected diffusion time step to an input sketch to simulate a partially noised image. We then used the reverse diffusion process to denoise this image, guiding the generation toward photorealistic outputs while preserving faithfulness to the original input. The pretrained model's denoising function was applied iteratively, gradually refining the noisy input into a realistic image. Our implementation follows the SDEdit framework, ensuring controlled and faithful edits without retraining the diffusion model. To measure faithfulness, we utilized L2 distance. For each output, we took the distance between it and the original sketch's image vector. This simple metric, also used in the original paper, was helpful to show that the borders, shapes, and colors remained faithful to the original sketch even after being passed through the model. For Realism, we used the same metric as the original paper, Kernel Inception Distance (KID). This uses a pre-trained inception model and a polynomial kernel in order to find features within realistic images, and then looks for those same features in the output.

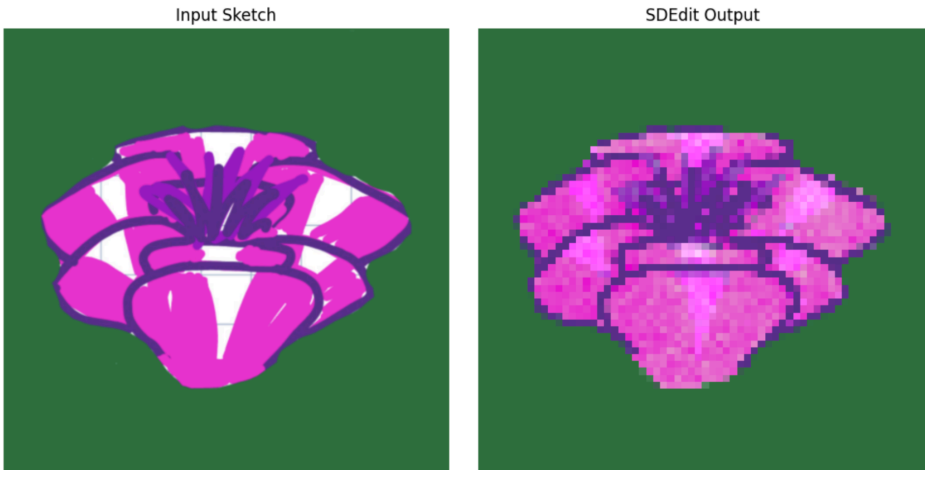
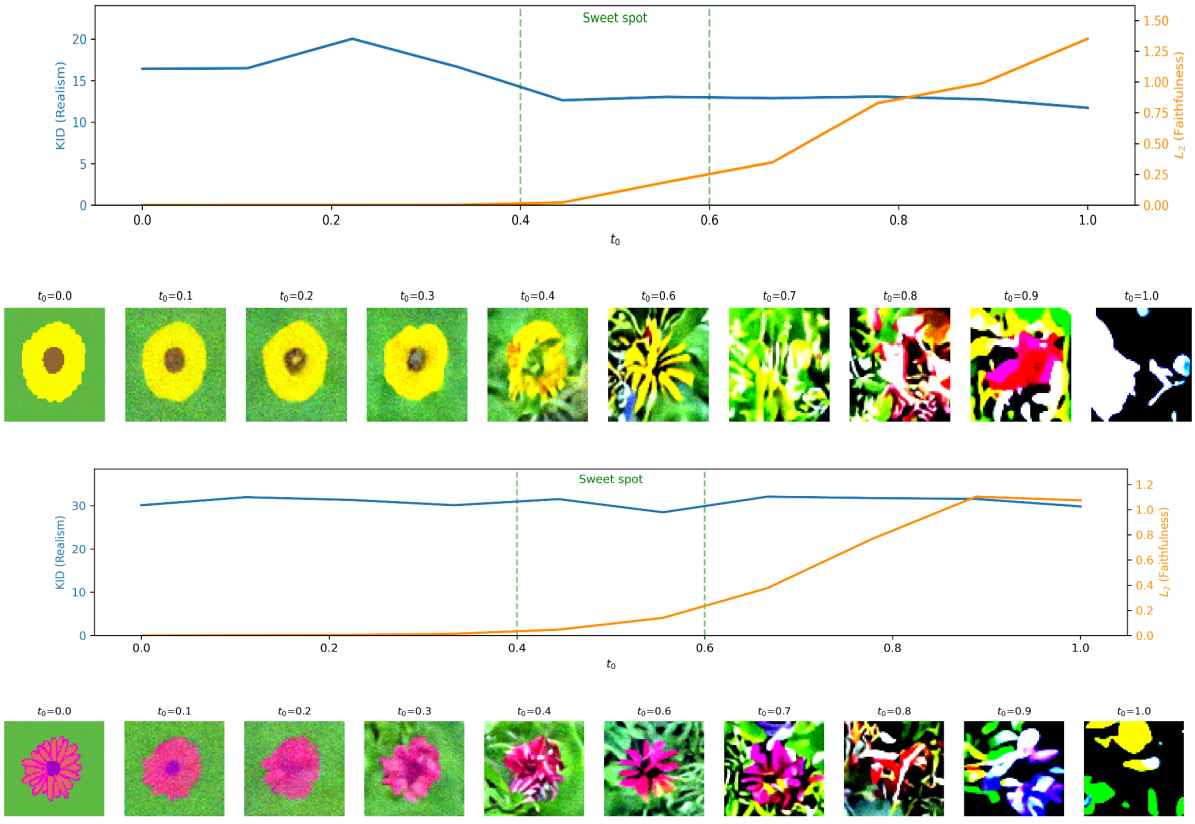



*Figure 2. Masked "edit" from sketch noised to t=0.4. Mask included background and petal outlines*

## Results:

Figures 3 and 4 display two examples of sketches outputs across different t0 values and the realism/faithfulness associated with them. In both instances, we saw the L2 Distance (Our Measure for Faithfulness) increase with higher t0, with values ranging from 0-0.25 in the sweet spot. This is intuitive as with higher t0, the output is likely to be less faithful to the original sketch. Conversely, we saw KID (Our Measure for Realism) fall as t0 increased. The sweet spot values differed by sketch, but ranged from 15-30. The sweet spot highlighted on our visuals is from 0.4-0.6, which is what was found in the original paper. Looking at our output images, it appears our sweet spot is closer to 0.3-0.4. This is likely due to a lack of resources for training and preparing the model, so it does not perform as well as the original when the image has more noise added.




*Figures 3 and 4. Faithfulness/Realism curve over varying levels of input noise. "Sweet Spot" is from 0.4-0.6, as stated in the original paper. (Faithfulness measured by L2 difference and Realism by KID)*

While our results did not match the original paper, they did show the Realism/Faithfulness tradeoff that is core to this problem. As increasing t0 allows more significant denoising at the beginning of our process, the final output deviates more from the original sketch (Shown by the increase in L2 distance). Conversely, if we do not denoise the image enough, we sacrifice realism in the output (Shown by the higher KID at lower t0 values).

Our output images, while far less realistic than the original paper, still clearly improved upon the sketches. With more resources for training and generation, we believe we could make these output images even more realistic.

## Challenges:

One issue we encountered in this process was evaluating the KID. L2 Distance was much simpler, as we were comparing the sketch to the output. When measuring KID, we are comparing the output to the real image we attempted to sketch, so the results are somewhat dependent on the quality of the sketch. Additionally, KID does not allow an image-to-image comparison and requires multiple outputs to compare to the one real image. We only utilized 3 samples because of the time it takes to create 10 edited images, but a lack of sample size could be another reason our realism curve is not smoother and clearly decreasing, as shown in the original paper.

There were a few other limitations we faced, however. Firstly, creating a diffusion model is incredibly computationally intensive, and our attempt at training one ourselves was completely unsuccessful due to computational limitations. Secondly, almost all pretrained diffusion models are published on the PyTorch architecture, and converting that to TensorFlow was far more difficult than expected. We were eventually able to find a working DDIM model in TensorFlow trained on the Oxford Flowers 102 database, which we were able to use for our SDEdit implementation.

## Reflection:

Overall, we think our project met its goals. We were able to use SDEdit to generate images and masked edits from sketches. Masking was one of our stretch goals, so getting it implemented was exciting. We would have liked our output images to be even more photo-realistic, especially around t0 = [0.45, 0.55], as that is where the original model performed the best. For us, that seems to be where the images got too noisy and began looking abstract and lost faithfulness to the original sketch.

Our model worked the way we expected it to. I think we knew going in our output images would be a little grainy/not as realistic as what we had read in the original paper. I think our goal was to produce a model that would behave correctly across the noising schedule, and produce output images that significantly improved realism from the original sketch. Given our results, we believe we did that.

We think the biggest changes in our approach over time were figuring out the best way to find a pre-trained diffusion model and begin using it. While this was just at the beginning of the project, finding a TensorFlow diffusion model we could get functional in our notebook was very difficult, and took several different methods and attempts to get it working. Another challenge was utilizing KID as a realism metric. Because KID requires large batches of images, and we only had the compute to train 3-4 samples, potentially pivoting to a different metric for realism could have been beneficial. One method we heard another group used was having GPT-4o look at the image, and have it decide whether it looks like a realistic flower. We could have used this method and had it decide realism on a scale from 0-10, which could have possibly given us a smooth sloping curve across the noising schedule.

If we had more time and resources, we would have generated far more image samples across the various noise levels; this would have let us calculate KID scores more reliably and get smoother, more consistent realism curves. With better compute, we could have also explored generating higher resolution outputs, which would have greatly improved the realism of our output images. More time would have also allowed us to experiment with a wider variety of input sketches, sketch edits of real images in

addition to full sketches.

One big takeaway we had was realizing how important it is to cater the framework to the task at hand. For working with diffusion models, PyTorch is definitely the better framework than Tensorflow; most of the resources, pretrained models, and community support are built around PyTorch, so trying to build it in TensorFlow ended up making things a lot harder than they needed to be. If I were to do this again, we would have definitely prioritized using the framework that fits the task.